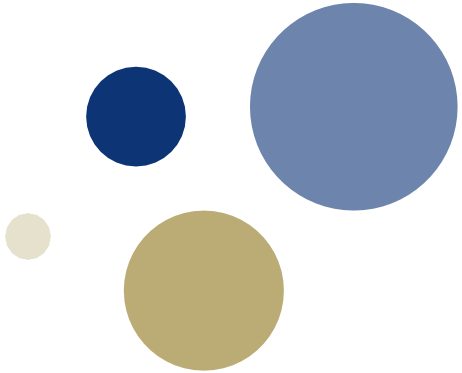
NTNU

Kunnskap for ei betre verd



Tupler, lister, arrays

Likheter, forskjeller, virkemåte, problemløsning

Guttorm Sindre, IDI
Dag Olav Kjellemo

Læringsutbytter, denne bolken

- Forstå tupler, lister, arrays
  - Hvorfor trenger vi dette?
  - Likheter og forskjeller
- Muterbare og immuterbare datatyper
  - Hva er forskjellen?
  - Muterende vs ikke-muterende operasjoner
- Iterering gjennom tupler, lister, arrays
  - Per element og per indeks
  - Én og flere dimensjoner
- Problemløsning med tupler, lister, arrays
  - Hvilken type når?

Hvorfor tupler, lister, arrays?

- Mange program gjør beregninger på mye data
  - F.eks. måleresultater inn fra sensorer
  - Data vi får tak i via nettet
  - …
- Med tusenvis av tall eller ord vi skal analysere
  - ekstremt tungvint med ny variabel for hver verdi, f.eks.
    - temp1 = 23.2
    - temp2 = 23.21
    - temp3 = 23.24
    - …
- Bedre om én variabel kan inneholde mange relaterte data:
  - temperaturer = [23.2, 23.21, 23.24, …]
  - maks_temp = max(temperaturer) # heller enn max(temp1, temp2, temp3, …)

Likheter

- **sekvenser** av data i rekkefølge
  - Én eller flere dimensjoner
- **indekseres** med [ ]
  - NB: alle tre bruker **[ ]** som parentes for indeksene
    - Selv om tuppel sine verdier omsluttes med (…, …)
  - Indeks starter på [0] forfra, [-1] bakfra
- **utdrag (slicing)** med [: :]

Opprette lister, tupler, arrays

Som for andre typer, tilordning:

```
L = []                              # blir ei tom liste
T = ()                              # blir et tomt tuppel
L = [5]                             # liste med ett element
T = (3,)                            # tuppel med ett element, MÅ ha komma her
L2 = L * 3                          # blir lista [5, 5, 5]
L = ['november', 3.3, 2.9, -1.2]    # liste med 4 element
T = ('Norge', 5.3, 47, 1, True)     # tuppel med 5 element
```

Opprettelse av array må bruke numpy-metoder (om ikke høyre side allerede er array)

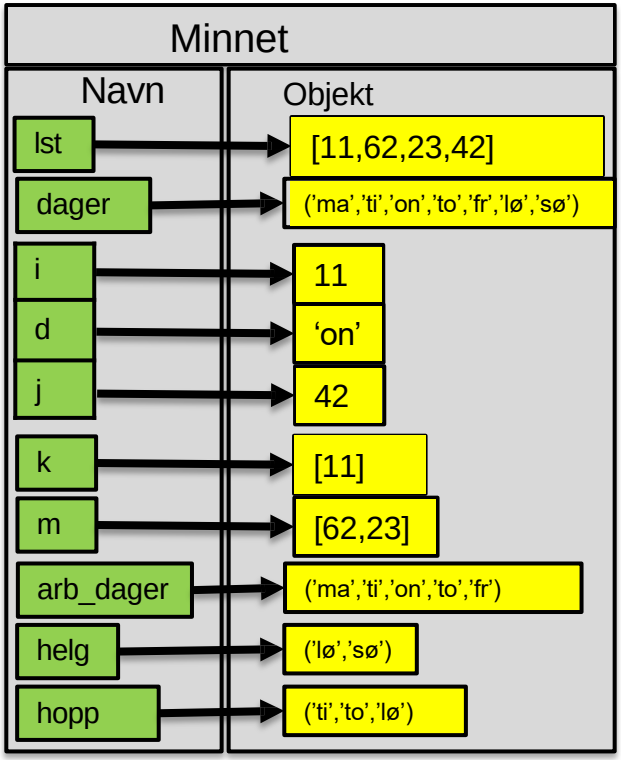
```
import numpy as np
A = [3.4, 2.1, 5.2]      # dette blir lista [3.4, 2.1, 5.2]
A = np.array(A)          # blir arrayet [3.4 2.1 5.2]
A2 = A * 2               # blir arrayet [6.8 4.2 10.4]
B = np.arange(1,4,0.5)   # blir [1.0 1.5 2.0 2.5 3.0 3.5]
C = np.linspace(1,5,51) # blir [1.0, 1.1, 1.2, ..., 5.0] (51 verdier totalt)
D = np.zeros(5)          # blir [0.0 0.0 0.0 0.0 0.0]
```

NB: Streng-visningen av array, f.eks. A -> [3.4 2.1 5.2] kan IKKE brukes i tilordning

Indeksering og «slicing»

- Eksempelkode, og hva som blir resultat i minnet
  - Kun eksempel liste og tuppel. Array samme prinsipp.

```
lst = [11, 62, 23, 42]
dager = ('ma','ti','on','to',
         'fr','lø','sø')
#enkeltelementer
i = lst[0]
d = dager[2]
j = lst[-1]
#utdrag (slicer)
k = lst[0:1]
m = lst[1:3]
arb_dager = dager[:5]
helg = dager[5:]
hopp = dager[1:6:2]
```



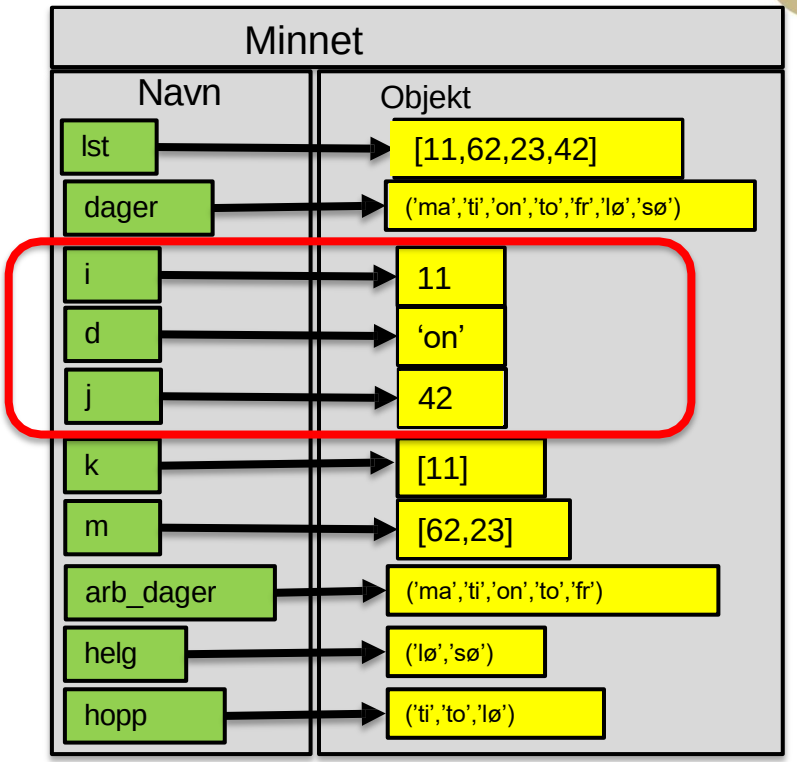
Kode: index_slice.py

Indeksering

- forfra: 0, 1, 2, … ; bakfra -1, -2, …

```
lst = [11, 62, 23, 42]
dager = ('ma','ti','on','to',
         'fr','lø','sø')
#enkeltelementer
i = lst[0]
d = dager[2]
j = lst[-1]
#utdrag (slicer)
k = lst[0:1]
m = lst[1:3]
arb_dager = dager[:5]
helg = dager[5:]
hopp = dager[1:6:2]
```

Kode: index_slice.py

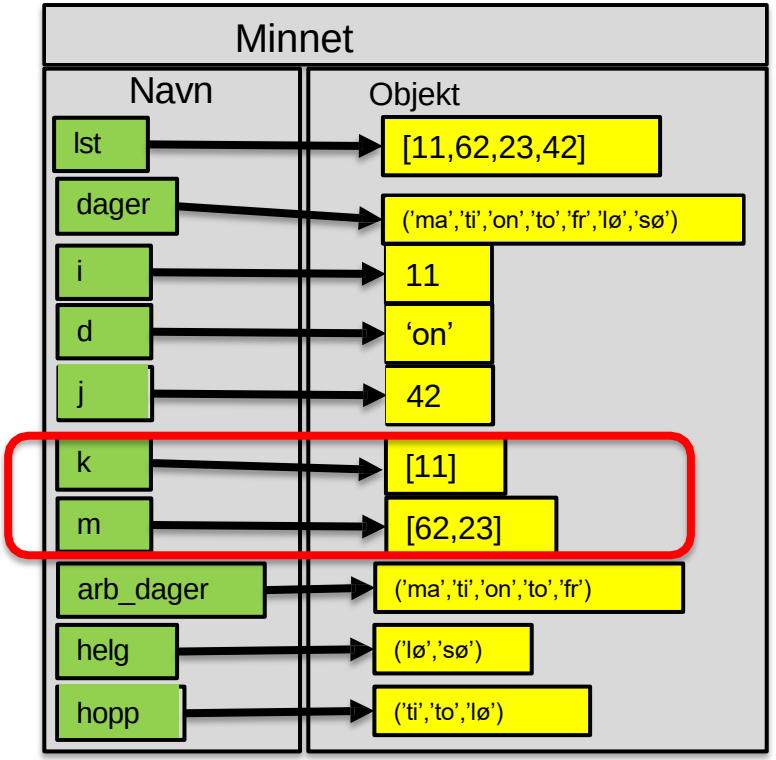


Utdrag («slicing»)

- [fra : til : steg] funker à la range(fra, til, steg)
  - Dvs. til er «til men **ikke** med»

```
lst = [11, 62, 23, 42]
dager = ('ma','ti','on','to',
         'fr','lø','sø')
#enkeltelementer
i = lst[0]
d = dager[2]
j = lst[-1]
#utdrag (slicer)
k = lst[0:1]
m = lst[1:3]
arb_dager = dager[:5]
helg = dager[5:]
hopp = dager[1:6:2]
```

Kode: index_slice.py

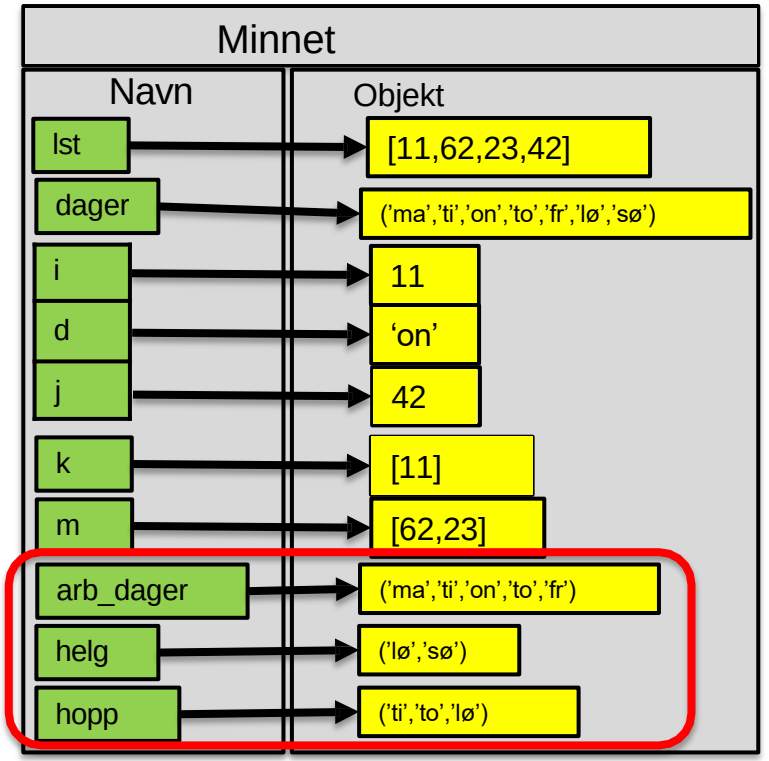


Utdrag (slicing), forts.

- fra er 0 hvis tom, til er «resten av lista» hvis tom
  - Stegverdi er 1 hvis ikke oppgitt

```
l  = [11, 62, 23, 42]
da er = ('ma','ti','on','to',
         'fr','lø','sø')
#  nkeltelementer
i  = lst[0]
d  = dager[2]
j  = lst[-1]
#  drag (slicer)
k  = lst[0:1]
m  = lst[1:3]
arb_dager = dager[:5]
helg = dager[5:]
hopp = dager[1:6:2]
```

Kode: index_slice.py



Forskjeller

	Tupler	Lister	Arrays
Del av standard Python	JA	JA	NEI (import numpy)
Uniforme datatyper	Valgfritt	Valgfritt	**Krav**
Uniforme dimensjoner	Valgfritt	Valgfritt	**Krav**
Massive regne-operasjoner for vektorer, matriser	NEI	NEI	**JA**
Hasjbar? (Eng. *Hashable*)	**JA**	NEI	NEI
Muterbar?	**NEI**	JA	JA
Endre størrelse?	NEI	**JA**	NEI

Forskjell: uniforme datatyper

- I lister og tupler kan vi ha blanding av datatyper
- *Lister* og *tupler* er samlinger av referanser til objekter, og ikke en samling av objekter.
- I et numpy-array **må** alle element ha samme datatype
  - Mest vanlig: alle er flyttall

```
formasjon = [4, 4, 2]                  # uniform datatype OK
temp = ['november', 3.3, 2.9, -1.2]    # blanding av typer OK
koord = (3.2, -2.7, 0.1)               # uniform datatype OK
land = ('Norge', 5.3, 47, 1, True)     # tuppel med 5 element
import numpy as np
xyz = np.array(koord)                  # Dette går bra, uniforme data
data = np.array(land)                  # blanding av typer -> alt gjøres om til tekst
```

```
[ 3.2 -2.7  0.1]
['Norge' '5.3' '47' '1' 'True']
```

Forskjell: uniforme dimensjoner

- I flerdimensjonale lister, tupler og array kan
  - Lister og tupler ha ulikt antall element i de indre sekvensene
  - Array må ha like mange element i de indre sekvensene

```
T2 = ( (3, 2), (1,) )    # 2D-tuppel, indre tupler har 2 og 1 element
L2 = [ [1, 2, 3] , [4, 5], [ ] ]  # 2D-liste, indre har 3, 2, 0 element
L3 = [ [1,2,3], [4,5,6], [7,8,9] ] # annen 2D-liste, uniform 3x3
import numpy as np
A2 = np.array(L2)   # ikke uniform, må lage array av lister, ikke av tall
A3 = np.array(L3)   # skal gå fint, uniforme dimensjoner
print(A2)
print(A3)
```

```
opprette_array.py:20: VisibleDeprecationWarning: Creating an ndarray from ragged nested sequences (which
is a list-or-tuple of lists-or-tuples-or ndarrays with different lengths or shapes) is deprecated. If you
meant to do this, you must specify 'dtype=object' when creating the ndarray.
  A2 = np.array(L2)   # trøbbel, ikke uniform
[list([1, 2, 3]) list([4, 5]) list([])]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Forskjell: effektive regneoperasjoner

```
T = tuple(range(5))     # gir (0, 1, 2, 3, 4)
L = [6, 6, 4, 2]
A = np.array(L)         # gir [6 6 4 2]
B = np.array(T)         # gir [0 1 2 3 4]

T2 = T * 2              # gir (0, 1, 2, 3, 4, 0, 1, 2, 3, 4)
L3 = L * 3              # gir [6,6,4,2,6,6,4,2,6,6,4,2]
A = A // 2              # gir [3 3 2 1]
B = B * 3 – 1           # gir [-1 2 5 8 11]
```

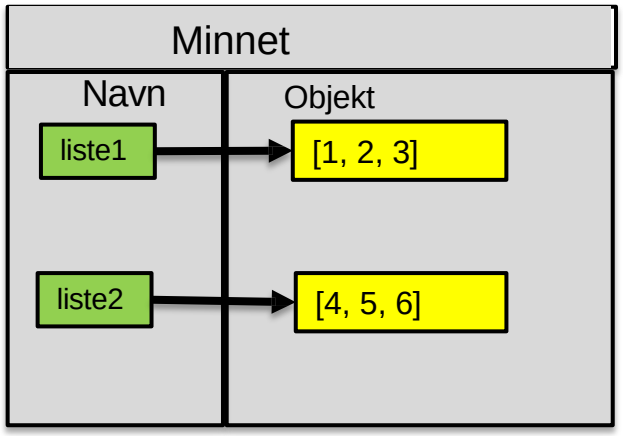
Forskjell: Muterbar vs. immuterbar

- Python: noen datatyper *muterbare*, andre *immuterbare*
  - Immuterbare, f.eks.: **int, float, bool, string, tuple**
  - Muterbare, f.eks.: **list, numpy.array, set, dict**
- Sier noe om **hvordan variable kan endres**
  - Muterbar: Både immuterende og muterende endringer mulig
  - Immuterbar: Kun immuterende endringer mulig
- Mutering: omtales gjerne som *«change in place»*
  - Dvs. endre innhold i et objekt *der det er* i minnet
  - (I motsetning til å redefinere variabel til å vise til annet objekt)

Hva er forskjellen?

- Immuterende endring: Gir variabelen en ny verdi
  - I figurer av navnerom/minne: omdirigerer pilen fra navnet
    - Endrer **ikke** objektet som variabelen viste til før
- Muterende endring: Endrer verdi *i* nåværende objekt
  - I figurer av navnerom/minne: endrer innhold i objektet
    - Flytter **ikke** pilen mellom navn og objekt

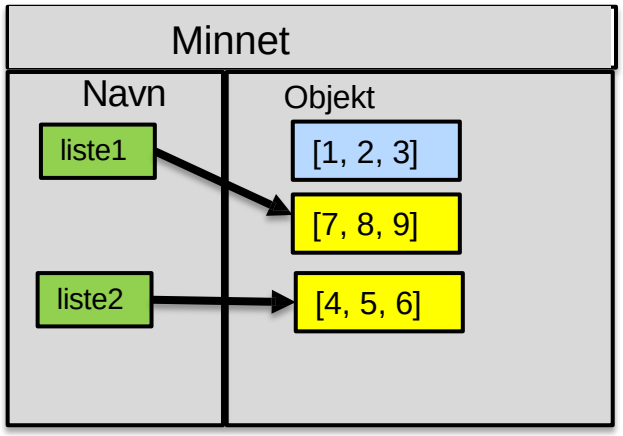
```
liste1 = [1,2,3]
liste2 = [4,5,6]
liste1 = [7,8,9]
liste2[0] = 13
```



Hva er forskjellen?

- Immuterende endring: Gir variabelen en ny verdi
  - I figurer av navnerom/minne: omdirigerer pilen fra navnet
    - Endrer **ikke** objektet som variabelen viste til før
- Muterende endring: Endrer verdi *i* nåværende objekt
  - I figurer av navnerom/minne: endrer innhold i objektet
    - Flytter **ikke** pilen mellom navn og objekt

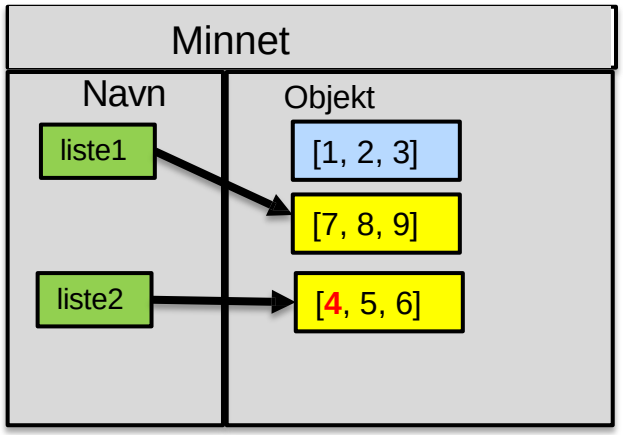
```
liste1 = [1,2,3]
liste2 = [4,5,6]
liste1 = [7,8,9]
liste2[0] = 13
```



Hva er forskjellen?

- Immuterende endring: Gir variabelen en ny verdi
  - I figurer av navnerom/minne: omdirigerer pilen fra navnet
    - Endrer **ikke** objektet som variabelen viste til før
- Muterende endring: Endrer verdi *i* nåværende objekt
  - I figurer av navnerom/minne: endrer innhold i objektet
    - Flytter **ikke** pilen mellom navn og objekt

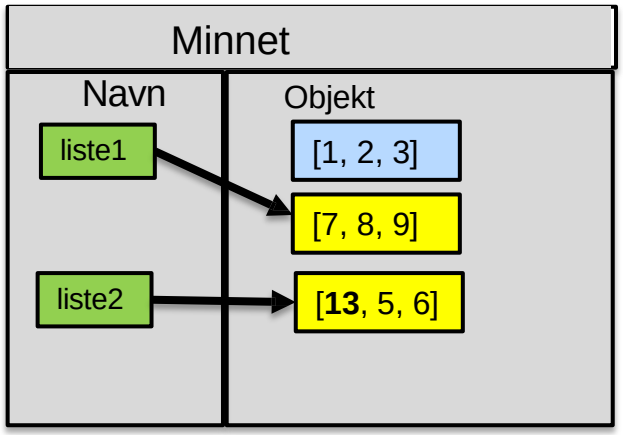
```
liste1 = [1,2,3]
liste2 = [4,5,6]
liste1 = [7,8,9]
liste2[0] = 13
```



Hva er forskjellen?

- Immuterende endring: Gir variabelen en ny verdi
  - I figurer av navnerom/minne: omdirigerer pilen fra navnet
    - Endrer **ikke** objektet som variabelen viste til før
- Muterende endring: Endrer verdi *i* nåværende objekt
  - I figurer av navnerom/minne: endrer innhold i objektet
    - Flytter **ikke** pilen mellom navn og objekt

```
liste1 = [1,2,3]
liste2 = [4,5,6]
liste1 = [7,8,9]
liste2[0] = 13
```



Muterende endringer *i* lister



- «change in place»
- Flere måter dette kan gjøres på:
  - Indeksering på *venstre side* i tilordning
  - Utvidede tilordningsoperatorer += og *=
  - Metoder som muterer lister

- Array tillater kun mutering som *endrer verdier*
  - IKKE mutering som fjerner eller legger til verdier
  - Fordi array ikke kan endre størrelse (in place)

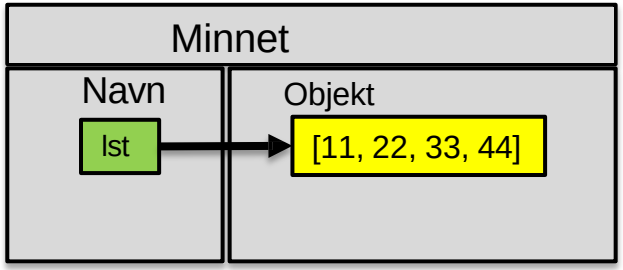
Muterende endring, liste og array

- «change in place»
- Flere måter dette kan gjøres på:
  - Indeksering på *venstre side* i tilordning
  - Utvidede tilordningsoperatorer += og *=
  - Metoder som muterer lister

Mutering ved indeksering

- Bruke indekser på *venstre side* av tilordning
- Én indeks: Endre ett element i lista
- Slice: Endre / skyte inn / fjerne deler av lista

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```

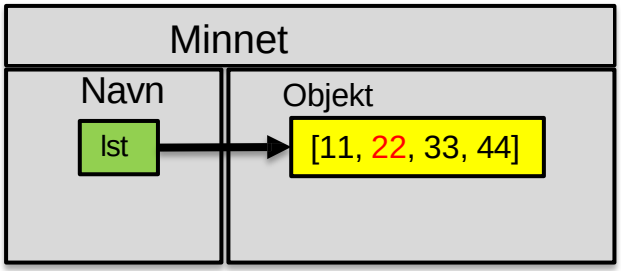


Kode: mutate_by_index.py

Mutering ved indeksering

- Bruke indekser på *venstre side* av tilordning
- Én indeks: Endre ett element i lista
  - Virker også for array

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```

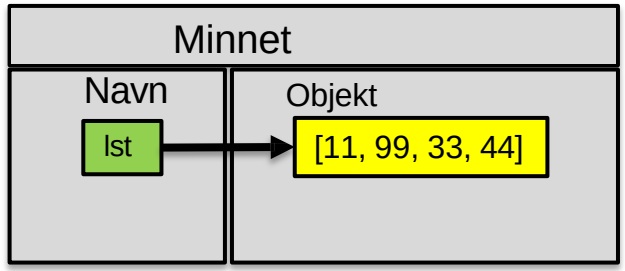


Kode: mutate_by_index.py

Mutering ved indeksering

- Bruke indekser på *venstre side* av tilordning
- Én indeks: Endre ett element i lista
  - Virker også for array

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```

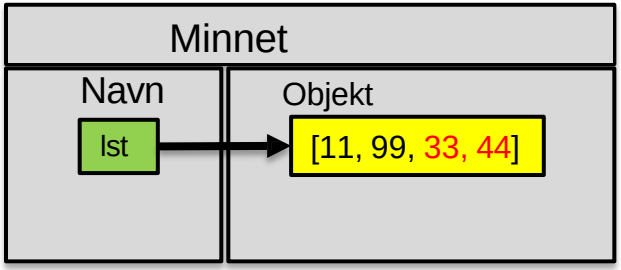


Kode: mutate_by_index.py

Mutering ved slice

- Bruke indekser på *venstre side* av tilordning
- Slice: Endre / skyte inn / fjerne deler av lista
  - IKKE for array

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```

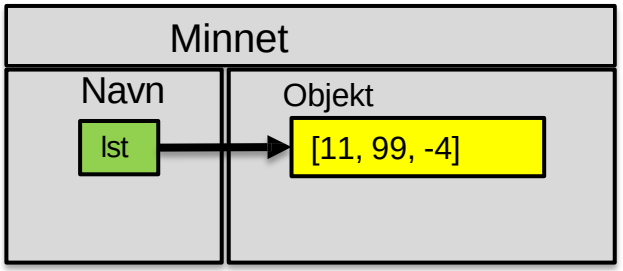


Kode: mutate_by_index.py

Mutering ved slice

- Slice: Endre / skyte inn / fjerne deler av lista
  - IKKE for array
    - Array kan ikke muteres på antall elementer
    - Bare på verdien til elementer

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```

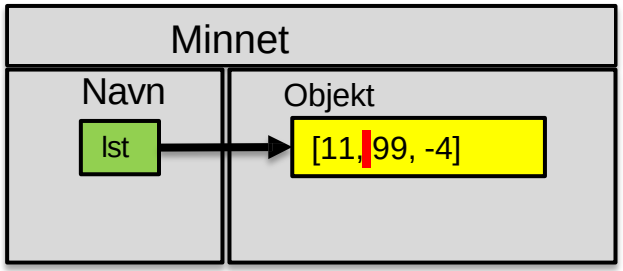


Kode: mutate_by_index.py

Endringer *i* lister (mutering)

- Slice: Endre / skyte inn / fjerne deler av lista
  - IKKE for array
    - Array kan ikke muteres på antall elementer
    - Bare på verdien til elementer

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```

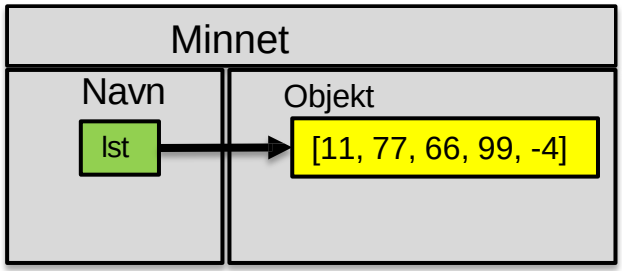


Kode: mutate_by_index.py

Endringer *i* lister (mutering)

- Slice: Endre / skyte inn / fjerne deler av lista
  - IKKE for array
    - Array kan ikke muteres på antall elementer
    - Bare på verdien til elementer

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```

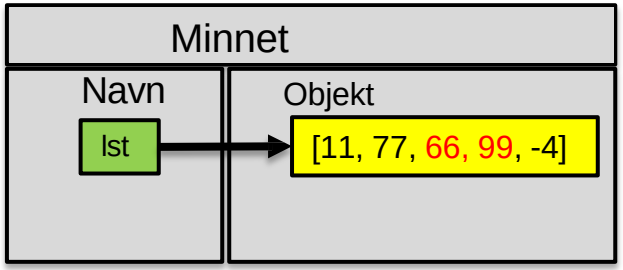


Kode: mutate_by_index.py

Endringer *i* lister (mutering)

- Slice: Endre / skyte inn / fjerne deler av lista
  - IKKE for array
    - Array kan ikke muteres på antall elementer
    - Bare på verdien til elementer

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```

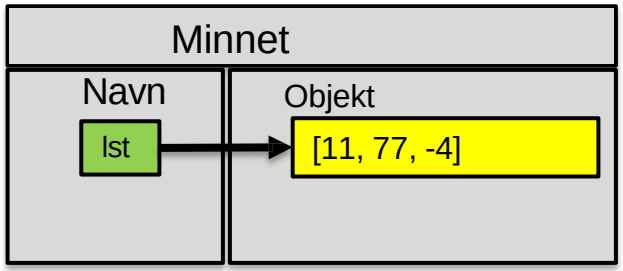


Kode: mutate_by_index.py

Endringer *i* lister (mutering)

- Slice: Endre / skyte inn / fjerne deler av lista
  - IKKE for array

```
# oppretter ei liste
lst = [11, 22, 33, 44]
# endrer enkeltelement:
lst[1] = 99
# endrer utdrag:
lst[2:] = [-4]
# skviser inn utdrag
lst[1:1] = [77, 66]
# fjerner utdrag
lst[2:4] = []
```



Kode: mutate_by_index.py

Hva skjer med immuterbare typer?

Strenger, tupler – forsøker indeksering på v.s. av =

- Dette gir feilmelding som vist nedenfor

```
>>> s = 'Hei'
>>> s[0] = 'N'
 Traceback (most recent call last):
   File "<pyshell>", line 1, in <module>
 TypeError: 'str' object does not support item assignment
>>> t = (11,22,33,44)
>>> t[0:0] = [0]
 Traceback (most recent call last):
   File "<pyshell>", line 1, in <module>
 TypeError: 'tuple' object does not support item assignment
```

- Men går fint for lister (lysarkene foran),
  - samt også for array hvis vi kun endrer en verdi,
  - Ikke fjerner eller legger til…

```
>>> A = np.arange(1.1, 1.7, 0.1)
>>> print(A)
 [1.1 1.2 1.3 1.4 1.5 1.6]
>>> A[-1] = 9.9
>>> print(A)
 [1.1 1.2 1.3 1.4 1.5 9.9]
```

Forskjell: endre størrelse

- Lister kan muteres ved å
  - Endre verdi for elementer
  - Legge til å fjerne elementer
- Arrays: kun endre verdi på elementer

```
A = np.arange(1.1, 1.7, 0.1)  # gir [1.1 1.2 1.3 1.4 1.5 1.6]
A[1] = 0.9                    # gir [1.1 0.9 1.3 1.4 1.5 1.6]
A[2:2] = [0.1, 8.8]           # går ikke (skvise inn elementer)
A.append(7.7)                 # går ikke (legge til element bakerst)
del A[2]                      # går ikke (fjerne element)
```

Iterere sekvenser med for-løkker

- To ulike måter å iterere på
  - Aksessere verdiene direkte
  - Aksessere via indeks
- Viktig å skjønne forskjellen!
  - Første eksempel: gjør samme på to ulike måter

```
liste = ['Per', 'Nina', 'Jo', 'Lucinda', 'Geir']
for navn in liste:
        print(navn)
# navn blir 'Per', så 'Nina', så 'Jo', ...
```

```
liste = ['Per', 'Nina', 'Jo', 'Lucinda', 'Geir']
for i in range(len(liste)):
        print(liste[i])
# i blir 0, 1, 2, 3, 4
# dermed blir liste[i] 'Per', 'Nina', 'Jo', ...
```

  - Andre eksempel: to ulike behov:

```
L = [5.32, 0.0, 5.67, 6.08, 0.0, 6.01]
lengste = L[0]
for x in L:   # x blir 5.32, 0.0, 5.67, ...
        if x > lengste:
                lengste = x
print('Lengste hopp:' , lengste)
```

Problemet krever **bare verdiene.**

```
L = [5.32, 0.0, 5.67, 6.08, 0.0, 6.01]
nr = 0 # indeks for lengste hopp
for i in range(len(L)): # range(6): i blir 0, 1, 2, 3, 4, 5
        if L[i] > L[nr]:
                nr = i
print('Lengste hopp:' , L[nr])
print(' kom i forsøk nr' , nr + 1)
```

Krever **posisjon i lista**

69

Iterere todimensjonale sekvenser

- Typisk med doble for-løkker
- Eksempel:
  - Vi har et 2D array med både positive og negative tall
  - Ønsker et array hvor oddetall er ganget med 2
  - mens partall forblir uendret
- To varianter
  - Vi trenger ikke de opprinnelige dataene
    - Kan mutere negative tall til null i opprinnelig array
  - Vi trenger de opprinnelige dataene
    - Men ønsker et nytt array med doblede verdier i tillegg

Løsninger på de to variantene

```python
import numpy as np

def odd_mult_2(arr):
    '''Får inn et array arr, muterer dette arrayet slik at alle
       oddetall blir ganget med to, mens partall forblir uendret'''
    for i in range(len(arr)):
        for j in range(len(arr[i])):
            if arr[i,j] % 2: # tall er odde
                arr[i,j] *= 2
    '''Denne funksjonen endrer i selve arrayet, trenger derfor ikke
       å returnere et resultat med return'''
```

muterende

Kode: odd_x_2.py

```python
import numpy as np

def odd_multed_2(arr):
    '''Får inn et array arr, lager et nytt array med samme verdier
       bare at oddetall er ganget med to, dette returneres'''
    new_arr = arr.copy() # lager en kopi av arrayet (samme data, nytt objekt)
    for i in range(len(new_arr)):
        for j in range(len(new_arr[i])):
            if new_arr[i,j] % 2:      # tall er odde
                new_arr[i,j] *= 2     # med liste måtte vi ha indeksert [i][j]
    return new_arr
    '''Denne funksjonen endrer ikke opprinnelig array, lager et nytt,
       dette må derfor returneres'''
```

ikke-muterende

Muterbarhet og funksjonskall

- En funksjon kan ha **parametre**:
  - Parametrene er **lokale variable**
- Ved funksjonskall så overføres **argumentene** til parametrene ved vanlig **tilordning**.
- Konsekvensene av detter er at:
  - En variabel som brukes som argument kan ikke bli tilordnet nye data i funksjonen
  - Hvis variabelen referer til immuterbare data, så kan ikke dataene endres
  - Hvis vi ønsker å endre verdien til en variabel som peker til immuterbare data, så må ny verdi returneres, og tilordnes variabelen

```
def doble(x):
    x *= 2
    return x

a = 1  # Immuterbar
print(f' doble({a}) = {doble(a)}')
print(f'Men a er fortsatt lik {a}')

a = doble(a)   # Tilordne ny verdi til a
print(f'a er doblet: a = {a}')
```

Muterbarhet og funksjonskall

- Hvis variabelen refererer til muterbare data, f.eks. en liste, så dataene endres i funksjonen
- Hva skjer hvis vi bruker x = x * 2 i stedet for x *= 2 ?

```
def doble(x):
    x *= 2
    return x

L = [1,2,3]  # Muterbar data
doble(L)     # returnerer den muterte listen som nå er [1,2,3,1,2,3]
print(L)     # L peker til den samme listen, [1,2,3,1,2,3]
```

Generere elementer med for-løkker

Se notatbøker for noen eksempler

Konklusjon

- Tupler, lister, arrays dekker ulike behov
  - Tupler: Data som vi ikke skal endre
  - Lister: Hvis behov for å
    - Endre, fjerne og legge til elementer
    - Blande ulike datatyper
  - Numpy: Hvis uniforme data (typisk tall), og behov for effektive regneoperasjoner
- Funksjoner som jobber på lister og arrays
  - Skal funksjonen mutere opprinnelig liste / array?
  - Eller lage en ny variant, men samtidig beholde gammel?
  - Dette gir typisk to alternative løsninger